UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona (FIB)

Generative Graph Databases for Building Structures via Visual Analysis with Large Language Models

HANG YAO

Thesis supervisor:
Enrique Romero Merino (Department of Computer Architecture)

Thesis co-supervisor:
Lluís Ortega Cerdà (Department of Architectural Design)

Master's Degree in Data Science

Master's Thesis

Facultat d'Informàtica de Barcelona (FIB)
Universitat Politècnica de Catalunya – BarcelonaTech

June 11, 2026

Abstract

The architectural domain relies heavily on the manual analysis of 2D floor plans, a process that is time-consuming, costly, and prone to inconsistency between annotators. While the visual verification of spatial correctness still requires a human expert, modern **Vision-Language Models (VLMs)** offer a credible pathway to automate the structural extraction that precedes it.

This thesis proposes an end-to-end pipeline that converts raster floor-plan images into structured 2D and 3D topological graphs using a locally hosted VLM (**Qwen3-VL**). The system decomposes the extraction problem into six focused sub-tasks — classification, aggregation analysis, apartment detailing, graph assembly, human-in-the-loop refinement, and 3D alignment — each driven by a carefully engineered prompt that combines an architect's profile, strong-signal enumeration, ordered decision procedures, and one-shot visual examples. A deterministic algorithm subsequently aligns vertical circulation elements (staircases and elevators) across floors to produce a cohesive 3D building representation.

The pipeline is applied to a corpus of *1,285* social-housing plans from three **Catalan institutional datasets** (IBAVI, IMPSOL, INCASOL). Since no ground-truth annotations exist for these datasets, validation rests on three complementary criteria: internal self-consistency and manual spot-checking. The resulting graph database is designed to feed downstream generative models capable of producing structurally valid residential layouts.

Keywords: Vision-Language Models, floor-plan analysis, architectural graphs, prompt engineering, social housing, in-context learning, human-feedback

Contents

Chapter 1

Introduction

1.1 Motivation

Residential architecture is one of the most data-intensive disciplines in civil engineering: every housing project generates dozens of CAD drawings, technical reports, and graphic catalogues that capture decades of design knowledge. In the specific context of Catalan social housing — promoted by public agencies such as **IBAVI** (*Institut Balear de l'Habitatge*), **IMPSOL** (*Institut Metropolità de Promoció de Sòl*), and **INCASOL** (*Institut Català del Sòl*) — this knowledge is dispersed across more than a thousand floor plans whose typological consistency makes them an ideal substrate for data-driven research.

Despite the volume of available material, the information remains effectively locked inside raster images. Existing computer-aided design tools require manual interpretation, and modern data-hungry generative models cannot consume JPG drawings directly: they need structured, machine-readable descriptions of rooms, dimensions, and topology. Producing these annotations by hand is significantly expensive (a single floor plan can take an experienced architect between ten and twenty minutes to label fully) and inconsistent between annotators.

The recent maturation of **Vision-Language Models** has changed the landscape. Open-weight architectures such as Qwen3-VL [1], LLaVA [2], and InternVL [3] now combine reliable optical character recognition with strong spatial reasoning, making it plausible to extract architectural semantics directly from an image through natural-language prompting. This thesis investigates whether such a workflow can replace manual annotation for housing data, opening the door to training generative models that automatically produce valid residential layouts.

1.2 Objectives

The primary objective of this thesis is to design, implement, and validate an automated, end-to-end pipeline capable of extracting structured 2D architectural graphs from unannotated raster floor plans of Catalan social housing (e.g., IBAVI, IMPSOL, INCASOL). Utilizing a locally hosted Vision-Language Model (VLM), the system is engineered to

translate unstructured visual layouts into formalized topological data (apartments and structural connections), thereby generating a robust dataset of residential topologies to serve as foundational training data for future generative architectural models.

To achieve this overarching goal, the research is decomposed into the following specific, verifiable sub-objectives:

1. **VLM Infrastructure and Deterministic Classification:** Build a reproducible inference framework wrapping a localized VLM (e.g., Qwen3-VL) optimized for constrained hardware. This includes engineering a deterministic classification mechanism utilizing greedy decoding to accurately categorize floor plans as aggregated or individual residential units.

2. **Few-Shot Visual Prompting Architecture:** Design a multi-stage, persona-driven prompt strategy that employs canonical one-shot and two-shot visual reference examples. This approach decomposes the topological extraction process into focused sub-tasks—classification, aggregation analysis, and apartment-level detailing—to systematically minimize visual hallucination.

3. **Topological Graph Assembly:** Translate the sequentially extracted visual features into formalized 2D graph structures (encoded via *JSON* and *NetworkX*), where nodes denote discrete apartments, individual rooms, and shared circulation elements (e.g., staircases, elevators), and edges represent physical spatial connections.

4. **Persistent Visual Memory (HITL):** Implement a human-in-the-loop (HITL) correction mechanism that captures expert architectural feedback. This system will extract reference crops of complex architectural elements and persist them as a dynamic, few-shot visual memory buffer across inference sessions, allowing the model to adapt without requiring weight-level retraining.

5. **Annotation-Free Validation Strategy:** Formulate and apply a self-consistency validation framework. By aggregating pipeline statistics and enforcing heuristic spatial logic rules, the system will evaluate the generated structural topologies autonomously, bypassing the reliance on manual ground-truth datasets, which are unavailable for the target architectural corpora.

1.3 Scope

The scope of this research is strictly bounded to Catalan and Spanish social-housing floor plans sourced from three institutional datasets (IBAVI, IMPSOL, INCASOL),

amounting to 1,285 raster images. Plans involving non-residential typologies, exterior site drawings, and pure elevations are explicitly filtered out during preprocessing.

The chosen Vision-Language Model is Qwen3-VL (specifically the 8B Instruct variant [4]). This model was selected for its open weights, native multi-image input processing—which is mandatory for few-shot visual prompting—and its state-of-the-art performance on document-understanding benchmarks. This latter capability is highly relevant given the density of architectural text abbreviations on social-housing plans. The model is executed locally and remains completely frozen. All engineering efforts are concentrated entirely on persona-driven prompt design, dynamic in-context visual examples, and pipeline orchestration. This constraint ensures the research remains fully reproducible on standard commodity hardware (e.g., an NVIDIA A5000 GPU for full-dataset inference) while bypassing the financial costs and algorithmic opacity associated with commercial cloud APIs.

The pipeline produces two specific artifacts: (i) structured, per-image 2D architectural graphs in JSON format, encompassing room typologies and physical topological connections, and (ii) per-building 3D spatial graphs that systematically link the single-floor 2D graphs through shared vertical circulation nodes. The downstream generation of novel floor plans utilizing this extracted dataset is explicitly out of scope; it is reserved as future work.

1.4 Document Structure

The remainder of this thesis is organized as follows:

- **Chapter 2** reviews the relevant state of the art, covering traditional floor-plan analysis, multi-modal language models, in-context learning strategies, and graph-theoretic representations in architecture.
- **Chapter 3** formalizes the problem of unstructured visual data in social housing and presents the high-level system architecture designed to address it.
- **Chapter 4** details the core methodological contributions, including the deterministic classification strategy, the multi-step 2D extraction prompts, the Human-in-the-Loop (HITL) persistent visual memory, and the deterministic algorithms utilized for 3D vertical alignment.
- **Chapter 5** documents the technical implementation of the VLM inference wrapper and the dataset processing pipeline.
- **Chapter 6** reports the experimental setup, detailing the self-consistency validation framework and the statistical results of both the 2D extraction and 3D

graph generation.

- **Chapter 7** analyzes the successes of the proposed pipeline while critically examining its limitations, particularly regarding visual hallucinations on dense blueprints.
- **Chapter 8** concludes the document with a summary of the technical contributions and outlines specific directions for future research.

Chapter 2

Background & State of the Art

2.1 Traditional Floor Plan Analysis

Computer-based interpretation of floor plans is a problem with a forty-year history. Early systems in the 1980s and 1990s focused on raster-to-vector conversion of CAD drawings, relying on heuristics such as Hough transforms to recover walls, doors, and dimensions [5]. These methods worked well on uniformly drafted technical drawings but degraded sharply on scanned, hand-drawn, or stylised plans.

The advent of convolutional neural networks shifted the field towards data-driven approaches. Liu et al. [6] introduced Raster2Vec, an end-to-end CNN that directly predicts a vector representation of walls and openings. Subsequent works such as CubiCasa5K [7] and the DeepFloorPlan family [8] expanded the taxonomy to include room semantics and furniture, achieving room-level pixel accuracies above 80% on curated benchmarks.

These approaches share three limitations that motivate the present work. First, they require sizable annotated datasets to generalise across visual styles — a resource that does not exist for Spanish/Catalan social-housing plans. Second, they output dense pixel masks rather than relational structures: the question *"is the kitchen connected to the living room?"* requires an additional post-processing step. Third, they cannot exploit textual cues such as room labels (*H1*, *EMC*, *cuina*) that are abundant in the source material.

2.2 Vision-Language Models

Multi-modal large language models combine a vision encoder with a transformer language model, enabling joint reasoning over images and text. The first notable open-weight family, LLaVA [2], demonstrated that a relatively small adapter between a CLIP [9] image encoder and a Vicuna [10] language model could produce coherent visual question answering. Subsequent architectures — BLIP-2 [11], InternVL [3], Qwen-VL [1] — progressively improved visual grounding, optical character recognition, and instruction-following.

The Qwen3-VL family [4], released in 2025, is particularly relevant to the present

work. It natively supports interleaved contexts of up to 256K tokens, seamlessly integrating text, images, and video. The 8B Instruct variant fits comfortably in 16 GB of GPU memory using `bfloat16` precision, which makes it deployable on common research hardware.

2.3 Prompt Engineering and In-Context Learning

Without fine-tuning, the behaviour of a VLM is governed entirely by the prompt. The literature on prompt engineering has identified several techniques that are directly applicable to architectural extraction.

Role conditioning. Giving the model in a specific role (*e.g.*, *"you are a professional architect specialised in Catalan social housing"*) has been shown to reduce hallucination on specialised vocabulary [12]. The persona biases the language model towards the appropriate sub-distribution of its training corpus.

Decision procedures. Decomposing a decision into an explicit ordered checklist — *"first count kitchens, then count bathrooms. . . "* — emulates the chain-of-thought paradigm [13] and reduces premature commitment to incorrect answers.

Few-shot and in-context learning. Prepending one or several reference examples (input + expected output) to the actual query emulates supervised fine-tuning at inference time [14]. For multi-modal models, the examples can be images with accompanying descriptions, a regime sometimes called *visual in-context learning* [15]. This thesis exploits both textual and visual in-context examples to teach the model the visual taxonomy of Spanish/Catalan social housing.

Deterministic decoding. For classification tasks with a small, fixed answer set, sampling-based decoding introduces unnecessary noise. Greedy decoding (`do_sample` = *False*) eliminates this noise and makes the partition fully reproducible across runs, an essential property for downstream reliability analysis.

2.4 Graph Representations for Architecture

A floor plan can be naturally encoded as a graph $G = (V, E)$ where vertices represent apartments or public elements and edges represent adjacency or connectivity. This formulation, sometimes called the *access graph* or *topological graph*, dates back to

Hillier's space-syntax theory [16] and remains the standard intermediate representation in computational architectural design.

Two graph variants are relevant. The *room-adjacency graph* encodes shared walls and is dense; the *access graph* encodes only door- and opening-mediated connections and is sparse, capturing the way inhabitants actually traverse the dwelling. The latter is more informative for generative modelling and is the representation adopted in this thesis.

Generative models for architectural layout fall into two families. The first operates on bitmaps using diffusion or GANs [17]: it produces visually plausible plans but with no guarantee of structural validity (*e.g.*, rooms may have no door). The second operates on graphs using GNN- or transformer-based architectures [18], taking a target topology as input and synthesising the corresponding geometry. The graphs produced by the present pipeline are designed to feed the second family of models.

Chapter 3

Problem Statement & System Architecture

3.1 The Problem

Given a dataset of $N = 1{,}285$ raster floor-plan images from three Sapnish/Catalan social-housing programmes, we wish to obtain, for each image, a structured representation suitable for downstream generative modelling. Formally, we seek a function

$$\phi : \mathcal{I} \longrightarrow \mathcal{G},$$

that maps an image $I \in \mathcal{I}$ to a graph $G \in \mathcal{G}$, where $\mathcal{G}$ is the space of architectural graphs whose nodes carry a type label (apartment, staircase, elevator) and whose edges denote physical accessibility.

Three properties of the problem make it non-trivial.

1. **Heterogeneous content.** A non-negligible fraction of the images correspond to *aggregation* plans (a full building floor showing several apartments around shared circulation), while the rest are *individual* plans of a single housing. These two categories require distinct extraction logic and must be disambiguated automatically.

2. **Visual diversity.** Plans vary in style (colour-filled, monochrome, CAD-rendered, hand-drawn), scale, and language of annotation (Catalan or Spanish). Classical computer-vision techniques struggle with this variability.

3. **Absence of ground truth.** No previous annotation effort exists for these datasets, ruling out supervised learning and forcing a validation strategy based on internal consistency, statistical plausibility, and human spot-checks.

3.2 Design Principles

To ensure the extraction pipeline remains robust, reproducible, and resistant to hallucination, four core design principles guide every subsequent engineering decision:

Task Decomposition. The extraction process is strictly modularized. No single Vision-Language Model (VLM) inference step attempts to parse the complete topological graph simultaneously. Instead, prompts are sequentially engineered so that the model evaluates a single, well-bounded architectural decision at a time (e.g., semantic zoning, followed by localized adjacency extraction).

Domain Anchoring. Every prompt initializes with an expert architectural persona and embeds a localized Spanish and Catalan abbreviation dictionary (e.g., *H*, *EMC*, *cuina*). This constraint forces the model to operate within the specific semantic subspace required to correctly interpret regional social-housing conventions.

Deterministic Execution. To maximize scientific reproducibility, rigid tasks such as binary classification utilize greedy decoding mechanisms (e.g., enforcing zero temperature and eliminating top-k sampling). Probabilistic sampling is strictly reserved for complex, open-ended extraction tasks where deterministic constraints would degrade the model's spatial reasoning capacity.

Diagnostic Preservation. Outputs that fail heuristic self-consistency validation are not discarded. Instead, they are archived alongside specific diagnostic flags. This fault-tolerant approach preserves critical failure states to directly inform the Human-in-the-Loop (HITL) visual memory mechanism and facilitate qualitative error analysis.

3.3 The Proposed Pipeline

The system architecture consists of a six-stage pipeline that systematically decomposes the visual extraction function, denoted as ϕ, into focused, modular sub-tasks. These sub-tasks are executed by a localized Vision-Language Model (VLM) utilizing tailored, stage-specific prompt architectures.

Step 1 — Classification. Each image is categorized as either *aggregation* or *individual* utilizing a deterministic, binary classification prompt. Two canonical reference images are injected as one-shot visual examples to strictly hold the model's visual taxonomy.

Step 2 — Aggregation Analysis. For multi-unit (aggregation) plans, the VLM identifies discrete apartments, staircases, and elevator shafts as distinct graph nodes, subsequently extracting the topological edges that connect them via shared circulation pathways.

Step 3 — Apartment Detailing. For each apartment node isolated in Step 2, the VLM is queried sequentially to extract internal room distributions and adjacencies, fully detailing the localized sub-graph.

Step 4 — Graph Assembly and Validation. The extracted architectural features are merged into a unified 2D spatial graph, normalized, and subjected to deterministic heuristic and structural logic checks. Graphs failing these self-consistency validation checks are flagged and retained for manual review.

Step 5 — Human-in-the-Loop Refinement. An interactive feedback module allows an expert architect to provide corrective textual hints and reference image crops. Rather than updating model weights, these corrections are persisted on disk as a dynamic visual memory, functioning as few-shot in-context learning examples for subsequent inference sessions.

Step 6 — 3D Alignment. Single-floor 2D graphs belonging to the same building are mathematically aligned along the Z-axis. This is achieved deterministically by linking vertical penetration nodes (e.g., staircases and elevators) across consecutive floors, yielding a cohesive multi-story 3D topology.

Chapter 4

Methodology

4.1 Data Partitioning

The combined corpus of *1,285* floor plans is partitioned, at runtime, by **Step 1** into the two semantic categories established in Section 3.1. The classifier is a Qwen3-VL prompt running in deterministic mode — greedy decoding with do_sample = False — which makes the partition fully reproducible across executions.

Two reference images are supplied as one-shot in-context examples: `inca_05.jpg` for the aggregation class and `inca_50.jpg` for the individual class, selected for their textbook-clear visual cues (six labelled apartments around a central core with staircase and elevators, versus a single dwelling with fully labelled Catalan room names).

4.2 Prompt Engineering Strategy

Because the VLM is never fine-tuned, the prompt is the only lever available to steer its behaviour. Four engineering techniques are applied consistently across all five VLM-driven steps of the pipeline.

4.2.1 Persona Conditioning

Every prompt opens with an architect persona, *e.g.*:

> *You are a professional architect specialised in residential building design and urban housing, with over fifteen years of experience reading and producing technical drawings for Catalan and Spanish social housing projects (IBAVI, IMPSOL, INCASOL).*

The persona biases the model towards the appropriate sub-distribution of its training data, treating domain-specific abbreviations (*H*, *EMC*, *SC*, *B*, *Te*, *P*) as familiar terminology rather than arbitrary tokens.

4.2.2 Strong-Signal Enumeration

Each prompt lists the unambiguous visual signals that characterise its target output. In the classification prompt, the aggregation class is described by signals such as repeated apartment identifiers (*1A*, *2B*, *HA*), multiple kitchens, and shared courtyards; the individual class by signals such as exactly one kitchen, exactly one bathroom, and the absence of any apartment identifier. These enumerations give the model concrete features to verify pixel-locally before committing to a label.

4.2.3 Decision Procedures

For high-stakes decisions (classification, vertical-circulation detection), the prompt embeds an explicit ordered checklist that the model is instructed to follow. For example, the classification prompt encodes the following routine:

1. Count the kitchens. If exactly one, return *individual*.
2. Count the bathrooms. If exactly one, return *individual*.
3. Look for repeated apartment identifiers. If none, return *individual*.
4. Only if multiple kitchens, multiple self-contained units, and a shared circulation element are present, return *aggregation*.

This emulates the chain-of-thought paradigm without requiring the model to verbalise intermediate reasoning, which would inflate output length.

4.2.4 Visual In-Context Examples

For tasks where visual judgement is decisive, the prompt is augmented with one or two reference images and their expected outputs. Step 1 receives two reference plans (one per class) with concise textual captions identifying their category. Step 2 receives a single canonical aggregation plan paired with its expected JSON output, teaching the model both the visual pattern and the response schema simultaneously. Step 5 receives zero, one, two, or three image crops loaded from a persistent directory of human-confirmed examples (see Section 4.5).

To avoid circular reference, the wrapper skips an example whenever the target image is the example itself.

4.3 2D Topological Extraction

For each image, the partition assigned in Step 1 determines the next call. Aggregation plans enter Step 2; individual plans bypass it and enter Step 3 directly.

4.3.1 Aggregation Analysis (Step 2)

The VLM emits a list of *nodes* (apartments, staircases, elevators) and *edges* (door- or landing-mediated connections). Apartments are anchored by their principal-entrance *door_position*, expressed as a percentage of image dimensions; staircases and elevators carry a centroid *position*. The one-shot reference described in Section 4.2.4 is provided to enforce both visual pattern and schema adherence.

4.3.2 Apartment Detailing (Step 3)

The output of Step 2 yields, for each aggregation plan, a list of apartments to inspect individually. The VLM is re-invoked with a localised prompt of the form *"focus only on the apartment labelled X, located approximately at* $[x, y] \ldots$ *"* and produces a JSON object encoding the room inventory: n_{bedrooms}, $n_{\text{bathrooms}}$, n_{kitchens}, $n_{\text{living rooms}}$, n_{terraces}, has_corridor, total_rooms, and the list of visible room labels. Individual plans are skipped.

4.4 Graph Assembly and Validation

The structured outputs of Steps 2 and 3 are merged into a 2D graph $G = (V, E)$ by Step 4, with apartment nodes carrying their internal inventory as a `details` attribute. The assembled graph is subjected to four deterministic structural checks.

Connectivity. All nodes must belong to a single connected component. Disconnected nodes are flagged.

Structural plausibility. Each apartment is required to contain at least one bedroom and one bathroom. Implausible combinations (*e.g.*, a bathroom that connects directly to a kitchen) raise warnings.

Duplicate detection. Nodes sharing both label and position within ϵ percent of image dimensions are merged.

Edge symmetry. Duplicate edges (u, v) and (v, u) are collapsed.

Failed checks are stored as `flags` inside the graph JSON, preserving the original output for human review rather than discarding it — the *diagnostic perservation* principle of Section 3.2.

4.5 Human-in-the-Loop Refinement

Even with careful prompting, the model occasionally fails to detect elements that an architect would identify immediately, particularly stair cores and elevator shafts drawn in unusual styles. Step 5 provides an interactive command-line tool that lets the architect inject two forms of feedback.

Textual hint. A free-text description of the missed element (*e.g.*, *"a staircase is present in the central core but was not detected"*) is interpolated into a re-extraction prompt that asks the model to re-examine the image with this knowledge.

Reference image crop. The architect may additionally supply a cropped image of the missed element (*e.g.*, the staircase symbol used throughout the dataset). The crop is copied to a persistent directory `outputs/rlhf_examples/` organised by element type (staircase or elevator). On every subsequent VLM invocation, up to three crops per type are re-loaded and prepended as few-shot visual references. This persists the correction across sessions without any model fine-tuning.

This design treats human feedback as a form of long-term visual memory: once an architect has annotated one staircase, the model is far more likely to recognise similar staircases across the rest of the dataset.

4.6 3D Spatial Alignment

A building is not a stack of independent floors but a connected three-dimensional structure: the staircase that lands on floor k must rise to floor $k+1$, and likewise for elevators. Step 6 makes this explicit.

Let $\mathcal{B} = \{G_0, G_1, \ldots, G_{F-1}\}$ be the set of per-floor aggregation graphs belonging to the same building, ordered by floor index. The 3D graph $G_{3D} = (V_{3D}, E_{3D})$ is constructed in three phases.

Phase 1 — Node globalisation. Each node $v \in V_f$ on floor f is assigned a global identifier

$$\mathrm{id}_{3D}(v) = f \times 10^6 + \mathrm{id}_{2D}(v),$$

chosen so that the floor index can be recovered by integer division and the intra-floor identifier by modulo.

Phase 2 — Horizontal-edge replication. For each floor f and each edge $(u, v) \in E_f$, an edge of type *horizontal* is added to E_{3D}, tagged with the floor index. These

edges encode the within-floor accessibility relationships extracted in Step 2.

Phase 3 — Vertical alignment. For every pair of consecutive floors $(f, f+1)$, vertical-circulation nodes are paired by spatial proximity. Let S_f and S_{f+1} denote the staircase node sets on floors f and $f+1$; a greedy nearest-neighbour matching is performed on their position vectors:

$$\text{match}(s_i, s_j^\star) \quad \text{such that} \quad s_j^\star = \underset{s_j \in S_{f+1} \setminus \text{used}}{\arg\min} \|\mathbf{p}(s_i) - \mathbf{p}(s_j)\|_2.$$

The same procedure is applied independently to the elevator node sets L_f and L_{f+1}. Each matched pair contributes one *vertical* edge tagged with the type of the circulation element. Unmatched extras — *e.g.*, a service staircase that exists on only some floors — are left disconnected and generate an `asymmetric_circulation` flag.

Post-conditions. The resulting graph is finally checked for global connectivity and for the presence of at least one vertical edge per floor pair. Buildings violating these conditions are flagged but retained.

4.7 Validation Strategy

The absence of ground truth precludes conventional metrics such as precision and recall. Validation in this thesis therefore relies on three complementary criteria, applied jointly to provide convergent evidence of correctness.

Internal consistency. The pipeline includes a self-verification prompt that re-shows the original image alongside the extracted graph and asks the model to flag rooms that do not exist, missing rooms, and incorrect connections. Disagreements between the original extraction and the self-verification step are recorded.

Manual spot-checks. A stratified random sample of 30–50 plans is reviewed by the author, who compares each extracted graph against the original image and records the rate of missing nodes, spurious nodes, incorrect connections, and incorrect room types. This yields an approximate accuracy estimate that, while not a substitute for ground truth, provides a credible lower bound on pipeline reliability.

Chapter 5

Implementation

This chapter documents the engineering decisions that translate the methodology of Chapter 4 into a reproducible software artefact.

5.1 Model Selection

Three critical constraints governed the selection of the Vision-Language Model (VLM): (i) *open weights*, ensuring scientific reproducibility and eliminating the financial and latency overhead of commercial cloud APIs across thousands of inference steps; (ii) *native multi-image support*, an absolute prerequisite for few-shot visual in-context learning; and (iii) *deployability within a 16 GB VRAM footprint*, matching the capacity of the remote NVIDIA RTX A5000 server available for full-dataset processing.

Within these rigid constraints, the Qwen3-VL family was prioritized over alternatives such as LLaVA, InternVL, and BLIP-2 for two primary reasons. First, Qwen3-VL demonstrates state-of-the-art performance on document-understanding benchmarks (e.g., DocVQA, ChartQA). This capability is paramount given the high density of textual abbreviations and dimensional annotations embedded within social-housing blueprints. Second, it natively processes interleaved multi-image inputs, whereas several competing architectures require the delicate and computationally inefficient concatenation of multiple images into a single visual canvas.

To facilitate both iterative development and large-scale extraction, two specific variants of the model were utilized:

Qwen3-VL-4B-Instruct ($\approx$ 8 GB in `bfloat16`) — utilized for local development, prompt engineering, and rapid prototyping on an NVIDIA RTX 2070 8 GB GPU, enabling efficient code iteration prior to server deployment.

Qwen3-VL-8B-Instruct ($\approx$ 16 GB in `bfloat16`) — deployed for the final production inference on the remote NVIDIA RTX A5000. Its expanded parameter capacity yields significantly higher reliability when extracting edge cases, such as plans with rotated text labels or partially occluded architectural elements.

A larger Mixture-of-Experts (MoE) variant, Qwen3-VL-30B-A3B-Instruct-FP8, was evaluated but ultimately rejected. Although its 3B active-parameter inference footprint

appears theoretically viable, the entire 30B parameter set must still reside in VRAM. Furthermore, FP8 precision compute is strictly restricted to Hopper and Ada Lovelace microarchitectures. The A5000 supports neither of these requirements, rendering the 30B MoE model untenable for this hardware environment.

5.2 Inference Infrastructure

The model is loaded through Hugging Face Transformers ($\geq$ 4.46) using the auto-class `AutoModelForImageTextToText` (the explicit `Qwen2_5_VLForConditionalGeneration` class produces a weight-shape mismatch on Qwen3-VL checkpoints and must be avoided). The `dtype` parameter is set to `torch.bfloat16`, balancing numerical accuracy and memory footprint.

A single `VLMClient` wrapper handles model loading (once per process) and exposes a single `query()` method. The method accepts an optional list of `example_images` (each a path-and-caption tuple) that are prepended to the prompt, and a `deterministic` boolean that switches between greedy and sampling decoding. This single abstraction is used uniformly across all six pipeline steps, isolating generation parameters from prompt content.

5.3 Hardware and Deployment

As introduced in Section 5.1, the inference pipeline was deployed across two distinct hardware environments to balance agile prototyping with large-scale processing capabilities:

Local development. A workstation running Windows 11 with an NVIDIA RTX 2070 (8 GB) hosts the 4B model. This environment is used for prompt iteration on small subsets (typically five to ten plans) and for human-in-the-loop refinement, which is inherently interactive.

Remote production. An Ubuntu server equipped with an NVIDIA A5000 (16GB) hosts the 8B model and runs the pipeline against the full 1,285-image corpus. Access is mediated by SSH and the Visual Studio Code Remote-SSH extension, which allows the same source tree to be edited locally and executed remotely without manual file transfers. Long-running batches are launched inside `tmux` sessions so that they survive disconnection.

5.4 Software Architecture

The codebase is organised around the six pipeline steps, with a thin orchestration layer and a small number of cross-cutting utilities.

- `src/config.py` — centralised configuration: paths, model identifiers, hyperparameters, and the canonical one-shot example images.
- `src/vlm_client.py` — the `VLMClient` wrapper.
- `src/step1_classify.py` through `src/step6_stack3d.py` — one module per pipeline step, each exposing both a single-image function and a batch function.
- `src/pipeline.py` — command-line orchestrator that composes the six steps and writes outputs to disk.
- `src/utils/json_parser.py` — robust JSON extractor that tolerates the additional prose VLMs sometimes emit before or after the JSON block.
- `src/utils/visualization.py` — NetworkX-based graph rendering for visual quality assurance.
- `prompts/*.txt` — prompt templates kept outside the code so that they can be edited without re-deploying the pipeline.

Outputs are written to a stable directory layout (`outputs/graphs`, `outputs/visualizations`, `outputs/logs`, `outputs/stats`, `outputs/rlhf_examples`), enabling incremental processing: re-runs only regenerate missing artefacts.

5.5 Reproducibility

Three measures guard reproducibility. First, all randomness in the classification step is eliminated by deterministic decoding. Second, the one-shot example images and the persisted HITL crops are committed alongside the source code, ensuring that any future re-execution sees the exact same in-context examples. Third, dependencies are pinned in `requirements.txt`, and the model checkpoint is loaded by exact revision hash rather than the moving `main` pointer of the Hugging Face Hub.

The pipeline can be reproduced end-to-end with a single command:

```
python -m src.pipeline --dataset INCASOL
```

which classifies every plan in the chosen dataset, extracts 2D graphs, applies validation, and writes the artefacts to `outputs/`.

Chapter 6

Experiments & Results

6.1 Experiment Setup

Chapter 7

Discussion & Limitations

7.1 Successes

7.2 Limitations

Chapter 8

Conclusion & Future Work

8.1 Summary of Contributions

8.2 Future Work

References

[1] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. *Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond*. 2023. arXiv: 2308.12966 [cs.CV]. URL: https://arxiv.org/abs/2308.12966.

[2] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. *Visual Instruction Tuning*. 2023.

[3] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, Bin Li, Ping Luo, Tong Lu, Yu Qiao, and Jifeng Dai. *InternVL: Scaling up Vision Foundation Models and Aligning for Generic Visual-Linguistic Tasks*. 2024. arXiv: 2312.14238 [cs.CV]. URL: https://arxiv.org/abs/2312.14238.

[4] Qwen Team. *Qwen3 Technical Report*. 2025. arXiv: 2505.09388 [cs.CL]. URL: https://arxiv.org/abs/2505.09388.

[5] Sheraz Ahmed, Marcus Liwicki, Markus Weber, and Andreas Dengel. "Improved Automatic Analysis of Architectural Floor Plans". In: Oct. 2011, pp. 864–869. DOI: 10.1109/ICDAR.2011.177.

[6] Chen Liu, Jiajun Wu, Pushmeet Kohli, and Yasutaka Furukawa. "Raster-to-Vector: Revisiting Floorplan Transformation". In: *2017 IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 2214–2222. DOI: 10.1109/ICCV.2017.241.

[7] Ahti Kalervo, Juha Ylioinas, Markus Häikiö, Antti Karhu, and Juho Kannala. *CubiCasa5K: A Dataset and an Improved Multi-Task Model for Floorplan Image Analysis*. 2019. arXiv: 1904.01920 [cs.CV]. URL: https://arxiv.org/abs/1904.01920.

[8] Zhiliang Zeng, Xianzhi Li, Ying Kin Yu, and Chi-Wing Fu. *Deep Floor Plan Recognition Using a Multi-Task Network with Room-Boundary-Guided Attention*. 2019. arXiv: 1908.11025 [cs.CV]. URL: https://arxiv.org/abs/1908.11025.

[9] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. arXiv: 2103.00020 [cs.CV]. URL: https://arxiv.org/abs/2103.00020.

[10] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. *Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality*. Mar. 2023. URL: https://lmsys.org/blog/2023-03-30-vicuna/.

[11] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. *BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models*. 2023. arXiv: 2301.12597 [cs.CV]. URL: https://arxiv.org/abs/2301.12597.

[12] Aobo Kong, Shiwan Zhao, Hao Chen, Qicheng Li, Yong Qin, Ruiqi Sun, Xin Zhou, Enzhi Wang, and Xiaohang Dong. *Better Zero-Shot Reasoning with Role-Play Prompting*. 2024. arXiv: 2308.07702 [cs.CL]. URL: https://arxiv.org/abs/2308.07702.

[13] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2023. arXiv: 2201.11903 [cs.CL]. URL: https://arxiv.org/abs/2201.11903.

[14] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. *Language Models are Few-Shot Learners*. 2020. arXiv: 2005.14165 [cs.CL]. URL: https://arxiv.org/abs/2005.14165.

[15] Yutong Bai, Xinyang Geng, Karttikeya Mangalam, Amir Bar, Alan Yuille, Trevor Darrell, Jitendra Malik, and Alexei A Efros. *Sequential Modeling Enables Scalable Learning for Large Vision Models*. 2023. arXiv: 2312.00785 [cs.CV]. URL: https://arxiv.org/abs/2312.00785.

[16] Bill Hillier and Julienne Hanson. *The Social Logic of Space*. Cambridge University Press, 1984.

[17] Nelson Nauata, Sepidehsadat Hosseini, Kai-Hung Chang, Hang Chu, Chin-Yi Cheng, and Yasutaka Furukawa. *House-GAN++: Generative Adversarial Layout Refinement Networks*. 2021. arXiv: 2103.02574 [cs.CV]. URL: https://arxiv.org/abs/2103.02574.

[18] Nelson Nauata, Kai-Hung Chang, Chin-Yi Cheng, Greg Mori, and Yasutaka Furukawa. *House-GAN: Relational Generative Adversarial Networks for Graph-constrained House Layout Generation*. 2020. arXiv: 2003.06988 [cs.CV]. URL: https://arxiv.org/abs/2003.06988.